

PYTHON PROGRAMMING

오프라인

2026.06.28 SUN

파이썬 프로그래밍 5일차 강의자료

파일 입출력 · 예외 처리 · 클래스 기초

오늘의 한 줄 목표

데이터를 저장하고, 오류에 버티며, 클래스로 구조화한다.

대상: 비전공자 1~4학년 · 구성: 이론 + 실습



오늘의 시간표

개념을 크게 잡고 바로 코드로 확인

교시	주제	이론	실습
1	파일 입출력	open·with·r/w/a	메모·일기
2	CSV & JSON	csv·json	성적·설정
3	예외 처리	try·raise	안전 계산기
4	클래스 ①	class·init·self	Student·Car
5	클래스 ②	메서드·객체 리스트	BankAccount

오늘의 전환

- 프로그램을 꺼도 데이터가 남습니다.
- 잘못된 입력에도 프로그램이 죽지 않습니다.
- 데이터와 기능을 클래스로 묶습니다.

TAKEAWAY

오늘의 목표: 저장되고, 오류에도 버티고, 구조화된 프로그램을 만든다.

복습

지난주 복습 & 오늘의 연결고리

개념을 크게 잡고 바로 코드로 확인

지난주까지

- 변수·조건·반복·자료구조·함수·문자열로 기초 문법을 완성했습니다.
- 하지만 프로그램을 끄면 데이터가 사라지고, 잘못된 입력에는 쉽게 멈춥니다.

오늘의 해결

- 파일 입출력: 데이터를 영구 저장
- 예외 처리: 오류에도 안 죽는 흐름
- 클래스: 데이터와 기능을 한 단위로 묶기

끝나면 만들 수 있는 것

TAKEAWAY

저장·안정성·구조화는 실전 프로그램으로 가는 세 갈래 문입니다.

1교시

1교시 — 파일 입출력 기초

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 파일을 열고 읽고 써서 데이터를 영구히 저장한다.

이론 20분

실습 30분

휴식 10분

이론

데이터가 사라지는 문제

개념을 크게 잡고 바로 코드로 확인

핵심 개념

- 어제 만든 성적 관리·단어 퀴즈... 끄면 싹 사라짐
- 변수·리스트는 메모리(RAM)에 잠깐 저장 → 종료 시 소멸

적용 질문

- 카톡 대화, 게임 세이브는 어떻게 남아있을까?
- 정답: 파일에 저장하기 때문!

TAKEAWAY

어제 만든 성적 관리·단어 퀴즈... 끄면 싹 사라짐

이론

파일이란?

개념을 크게 잡고 바로 코드로 확인

핵심 개념

- 파일 = 디스크(저장장치)에 영구 보관되는 데이터 덩어리
- 텍스트 파일(.txt), 표 파일(.csv), 사진(.jpg), 문서(.docx)...

적용 질문

- 프로그램이 끝나도 남아있음 → 다음에 다시 불러오기 가능
- 오늘은 가장 기본인 텍스트 파일부터!

TAKEAWAY

파일 = 디스크(저장장치)에 영구 보관되는 데이터 덩어리

이론

파일 다루는 3단계

개념을 크게 잡고 바로 코드로 확인

핵심 개념

- 비유: 냉장고 열기 → 꺼내거나 넣기 → 닫기
- 안 닫으면? 데이터가 제대로 저장 안 될 수 있음
- 열기(open) — 파일을 사용할 준비
- 작업(read/write) — 읽거나 쓰기
- 닫기(close) — 파일 정리

적용 질문

TAKEAWAY

비유: 냉장고 열기 → 꺼내거나 넣기 → 닫기

이론

open() — 파일 열기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- open(파일이름, 모드) 로 파일을 엽
- 결과를 변수(f)에 저장해 사용
- 작업 후 반드시 close()

Python example

```
f = open("memo.txt", "w") # 파일 이름, 모드  
f.write("안녕하세요")  
f.close()
```

TAKEAWAY

open(파일이름, 모드) 로 파일을 엽

이론

파일 모드 — r, w, a

개념을 크게 잡고 바로 코드로 확인

모드	의미	설명
"r"	읽기(read)	파일 내용 읽기 (기본 값)
"w"	쓰기(write)	새로 쓰기 (기존 내용 삭제!)
"a"	추가(append)	기존 뒤에 이어 쓰기

읽는 법

- "w" 는 기존 파일을 덮어씀 → 주의!

TAKEAWAY

"w" 는 기존 파일을 덮어씀 → 주의!

이론

쓰기 모드 — write()

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- write() 로 내용을 파일에 기록
- encoding="utf-8" → 한글 깨짐 방지 (꼭 !)
- \n 으로 줄바꿈

Python example

```
f = open("diary.txt", "w", encoding="utf-8")
f.write("오늘은 파이썬을 배웠다.\n")
f.write("파일 저장도 배웠다.\n")
f.close()
```

TAKEAWAY

write() 로 내용을 파일에 기록

이론

close()의 중요성

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 파일을 닫아야 변경 내용이 확정 저장
- 깜빡하기 쉬움 → 다음 슬라이드의 with로 해결!

Python example

```
f = open("memo.txt", "w")  
f.write("중요한 데이터")  
# close()를 안 하면? → 저장이 안 될 수 있음!  
f.close() # 반드시!
```

TAKEAWAY

파일을 닫아야 변경 내용이 확정 저장

이론

with 문 — 자동으로 닫기 ★

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- with open(...) as f: 형태
- 블록이 끝나면 자동으로 닫힘 → close 깜빡 걱정 없음
- 앞으로는 항상 with 사용 권장!

Python example

```
with open("memo.txt", "w", encoding="utf-8")  
    as f:  
    f.write("안녕하세요")  
# 블록을 벗어나면 자동으로 close()!
```

TAKEAWAY

with open(...) as f: 형태

이론

with 문법 자세히

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 들여쓰는 블록 안에서만 파일 작업
- 블록을 나가면 자동 정리

Python example

```
with open("data.txt", "w", encoding="utf-8")
    as f:
    f.write("첫 줄\n")
    f.write("둘째 줄\n")
    # 이 안에서만 f 사용 가능
print("저장 완료!") # with 밖 = 이미 닫힘
```

TAKEAWAY

들여쓰는 블록 안에서만 파일 작업

이론

읽기 모드 — read()

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- "r" 모드 + read() → 파일 전체 내용을 문자열로
- 읽은 내용을 변수에 담아 활용

Python example

```
with open("diary.txt", "r", encoding="utf-8")  
    as f:  
    content = f.read()  
    print(content)
```

TAKEAWAY

"r" 모드 + read() → 파일 전체 내용을 문자열로

이론

한 줄씩 읽기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 파일을 for 로 돌리면 한 줄씩 읽음
- strip() 으로 끝의 \n 제거 (3일차 복습!)

Python example

```
with open("diary.txt", "r", encoding="utf-8")  
    as f:  
    for line in f:  
        print(line.strip()) # strip으로 줄바꿈 제거
```

TAKEAWAY

파일을 for 로 돌리면 한 줄씩 읽음

이론

readlines() — 줄들을 리스트로

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- readlines() → 각 줄을 리스트 요소로
- 줄 개수 세기, 특정 줄 접근에 유용

Python example

```
with open("diary.txt", "r", encoding="utf-8")
    as f:
    lines = f.readlines()
    print(lines)      # ['첫 줄\n', '둘째 줄\n']
    print(len(lines)) # 줄 개수
```

TAKEAWAY

readlines() → 각 줄을 리스트 요소로

이론

추가 모드 — a

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- "a" 모드 → 기존 내용 유지하고 뒤에 추가
- 일기.로그처럼 계속 쌓는 데이터에 적합
- "w" 였다면 기존 내용이 다 지워졌을 것!

Python example

```
with open("log.txt", "a", encoding="utf-8") as  
    f:  
    f.write("새 기록 추가\n")
```

TAKEAWAY

"a" 모드 → 기존 내용 유지하고 뒤에 추가

이론

파일이 없으면?

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 읽기(r) 모드인데 파일이 없으면 에러!
- 쓰기(w)·추가(a) 모드는 파일을 자동 생성
- 이런 에러 대응은 3교시 예외 처리에서!

Python example

```
with open("없는파일.txt", "r") as f: #  
    FileNotFoundError  
    content = f.read()
```

TAKEAWAY

읽기(r) 모드인데 파일이 없으면 에러!

이론

파일 경로 (간단히)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 파일 이름만 쓰면 현재 작업 폴더 기준
- 폴더 안이면 폴더/파일 형태
- VS Code에서 연 폴더가 기준점

Python example

```
open("memo.txt")      # 현재 폴더의  
                        memo.txt  
open("data/memo.txt")  # data 폴더 안
```

TAKEAWAY

파일 이름만 쓰면 현재 작업 폴더 기준

정리

파일 입출력 정리

개념을 크게 잡고 바로 코드로 확인

오늘의 결과물

- 3단계: 열기(open) → 작업(read/write) → 닫기(close)
- with 쓰면 닫기 자동! (항상 with)
- 모드: r(읽기) w(새로쓰기·덮어씀) a(추가)
- 한글은 encoding="utf-8", 줄바꿈은 \n

다음 연결

- for로 한 줄씩 읽기

TAKEAWAY

데이터는 저장되고, 오류에도 버티고, 코드는 사물처럼 구조화됩니다.

실습

메모 저장 & 불러오기

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
# 저장하기
memo = input("메모 내용: ")
with open("memo.txt", "w", encoding="utf-8")
    as f:
        f.write(memo)
print("저장 완료!")
# 불러오기
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

계속 쌓이는 일기장

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
while True:
    line = input("일기(끝: q): ")
    if line == "q":
        break
    with open("diary.txt", "a",
              encoding="utf-8") as f:
        f.write(line + "\n")
# ... 계속
```

실습 포인트

- "a" 모드로 계속 추가, 마지막에 전체 출력

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

파일 다루기 (택1 이상)

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

따라 하기

- 할 일 저장기: 할 일을 입력받아 파일에 한 줄씩 추가
- 줄 수 세기: 파일을 읽어 총 몇 줄인지 출력 (readlines)

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- 단어 검색: 파일에서 특정 단어가 든 줄만 출력 (in)
- 명단 저장: 이름 여러 개 입력받아 파일로 저장 후 다시 읽기

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

파일을 저장하고 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: CSV & JSON — 표·구조화 데이터를 파일로!

작성 파일 저장

에러 메시지 캡처

입력값 실험

다음 실습 준비

TAKEAWAY

파일 실습 전후에는 생성된 파일과 현재 폴더를 꼭 확인합니다.

2교시

2교시 — CSV & JSON

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 실제 데이터 형식인 CSV·JSON을 읽고 쓴다.

이론 20분

실습 30분

휴식 10분

이론

표 데이터를 저장하려면?

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 성적표처럼 표 형태 데이터를 텍스트로 저장하려면?
- 콤마로 칸을 구분한 이 형식이 바로 CSV!
- 엑셀로도 열리는 만능 데이터 형식

Python example

```
이름,점수  
홍길동,90  
김철수,85
```

TAKEAWAY

성적표처럼 표 형태 데이터를 텍스트로 저장하려면?

이론

CSV란?

개념을 크게 잡고 바로 코드로 확인

핵심 개념

- CSV = Comma-Separated Values (콤마로 구분된 값)
- 한 줄 = 한 행(레코드), 콤마 = 칸 구분

적용 질문

- 엑셀, 구글시트, 데이터 분석에서 표준
- 가볍고 어디서나 열림

TAKEAWAY

CSV = Comma-Separated Values (콤마로 구분된 값)

이론

CSV를 직접 다뤄보면

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 직접 콤마 찍어도 되지만 번거롭고 실수 위험
- 그래서 전용 도구 csv 모듈 사용!

Python example

```
# 그냥 문자열로도 가능하지만...  
with open("scores.csv", "w", encoding="utf-8")  
    as f:  
    f.write("이름,점수\n")  
    f.write("홍길동,90\n")
```

TAKEAWAY

직접 콤마 찍어도 되지만 번거롭고 실수 위험

이론

csv 모듈 — 쓰기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- csv.writer → writerow() 로 한 줄씩
- newline="" → 빈 줄 생기는 것 방지

Python example

```
import csv

with open("scores.csv", "w", newline="",
        encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["이름", "점수"]) # 한 행 쓰기
    writer.writerow(["홍길동", 90])
    writer.writerow(["김철수", 85])
```

TAKEAWAY

csv.writer → writerow() 로 한 줄씩

이론

csv 모듈 — 읽기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- csv.reader → for로 한 행씩 리스트로 읽음
- 읽은 숫자는 문자열 → 계산하려면 int() 변환

Python example

```
import csv

with open("scores.csv", "r", encoding="utf-8")
    as f:
        reader = csv.reader(f)
        for row in reader:
            print(row)    # ['이름', '점수'],
                           ['홍길동', '90']...
```

TAKEAWAY

csv.reader → for로 한 행씩 리스트로 읽음

이론

여러 행 한 번에 쓰기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- writerows (복수형) → 여러 행을 한꺼번에
- 어제 배운 2차원 리스트가 그대로 쓰임!

Python example

```
import csv
data = [["이름", "점수"], ["홍길동", 90], ["김철수", 85]]

with open("scores.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(data) # 리스트의 리스트를 한 번에!
```

TAKEAWAY

writerows (복수형) → 여러 행을 한꺼번에

이론

CSV 활용 — 성적 처리 흐름

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- next(reader) 로 헤더 한 줄 건너뛰기

Python example

```
import csv
total = 0
count = 0
with open("scores.csv", "r", encoding="utf-8")
    as f:
        reader = csv.reader(f)
        next(reader)      # 헤더 줄 건너뛰기
        for name, score in reader:
            total += int(score)
# ... 계속
```

TAKEAWAY

next(reader) 로 헤더 한 줄 건너뛰기

이론

CSV의 첫 줄 — 헤더(머리글)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 보통 첫 줄은 열 이름(헤더), 둘째 줄부터 데이터
- 데이터만 처리할 땐 `next(reader)` 로 헤더를 건너뛴
- 엑셀에서 열면 자동으로 표의 제목 줄로 보임

Python example

```
이름,점수    ← 헤더(열 이름)
홍길동,90    ← 실제 데이터
김철수,85
```

TAKEAWAY

보통 첫 줄은 열 이름(헤더), 둘째 줄부터 데이터

이론

CSV 다룰 때 주의점

개념을 크게 잡고 바로 코드로 확인

핵심 개념

- 한글: encoding="utf-8" 안 쓰면 깨짐
- 빈 줄: 쓰기 모드에 newline="" (빈 줄 방지)

적용 질문

- 숫자: 읽으면 전부 문자열 → int()/float() 변환 필요
- 줄바꿈: csv 모듈이 알아서 처리 (직접 \n 불필요)

TAKEAWAY

한글: encoding="utf-8" 안 쓰면 깨짐

이론

JSON이란?

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- JSON = 구조화된 데이터 형식 (키-값 구조)
- 웹.앱에서 데이터 주고받는 표준
- 어디서 많이 본 모양? → 딕셔너리!

Python example

```
{  
  "name": "홍길동",  
  "age": 24,  
  "hobbies": ["독서", "코딩"]  
}
```

TAKEAWAY

JSON = 구조화된 데이터 형식 (키-값 구조)

이론

JSON ≈ 딕셔너리

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- JSON과 딕셔너리는 모양이 거의 같음
- 그래서 파이썬에서 다루기 아주 쉬움
- 중첩(리스트·딕셔너리 안에 또) 구조도 OK

Python example

```
# 파이썬 딕셔너리
{"name": "홍길동", "age": 24}

# JSON (거의 똑같음!)
{"name": "홍길동", "age": 24}
```

TAKEAWAY

JSON과 딕셔너리는 모양이 거의 같음

이론

json 모듈 — 저장(dump)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- json.dump(데이터, 파일) 로 저장
- ensure_ascii=False → 한글 그대로, indent → 들여쓰기

Python example

```
import json
data = {"name": "홍길동", "age": 24, "scores":
       [90, 85]}

with open("profile.json", "w",
          encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False,
              indent=2)
```

TAKEAWAY

json.dump(데이터, 파일) 로 저장

이론

json 모듈 — 불러오기(load)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- json.load(파일) → 딕셔너리로 복원
- 저장한 구조 그대로 (숫자는 숫자, 리스트는 리스트)

Python example

```
import json

with open("profile.json", "r",
        encoding="utf-8") as f:
    data = json.load(f)

print(data["name"])    # 홍길동
print(data["scores"])  # [90, 85] (리스트 그대로!)
```

TAKEAWAY

json.load(파일) → 딕셔너리로 복원

이론

dumps / loads (문자열 버전)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- dumps/loads (s = string) → 파일 없이 문자열로 변환
- 웹 통신에서 자주 (마지막 주 API와 연결)

Python example

```
import json
data = {"name": "홍길동"}
text = json.dumps(data, ensure_ascii=False) #
    딕셔너리 → 문자열
print(text)
    # {"name": "홍길동"}
back = json.loads(text)
    # 문자열 → 딕셔너리
print(back["name"])
    # 홍길동
```

TAKEAWAY

dumps/loads (s = string) → 파일 없이 문자열로 변환

이론

CSV vs JSON — 언제?

개념을 크게 잡고 바로 코드로 확인

	CSV	JSON
모양	표(행/열)	키-값(중첩 가능)
적합	성적표·엑셀 데이터	설정·프로필·복잡한 구조
형변환	필요(전부 str)	불필요(타입 유지)
친구	엑셀, pandas	웹 API, 앱

읽는 법

- 핵심 개념을 예시로 먼저 이해합니다.
- 코드로 직접 확인하며 감을 잡습니다.

TAKEAWAY

핵심 개념을 예시로 먼저 이해합니다.

실습

성적 csv 만들고 평균 내기

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
import csv
# 1) 저장
data = [{"이름", "점수"}, ["홍길동", 90], ["김철수",
85], ["이영희", 95]]
with open("scores.csv", "w", newline="",
encoding="utf-8") as f:
    csv.writer(f).writerows(data)
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

JSON 설정 파일

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
import json
# 설정 저장
settings = {"volume": 70, "theme": "dark",
            "language": "ko"}
with open("settings.json", "w",
          encoding="utf-8") as f:
    json.dump(settings, f, ensure_ascii=False,
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

CSV/JSON (택1 이상)

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

따라 하기

- 장바구니 CSV: 상품명·가격을 CSV로 저장 후 총액 계산
- 연락처 JSON: {이름: 번호} 딕셔너리를 JSON으로 저장·불러오기

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- CSV 최고점: 성적 CSV 읽어 1등 이름·점수 출력
- 게임 세이브: 레벨·점수·아이템(리스트)을 JSON으로 저장

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

휴식

휴식 10분

파일을 저장하고 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 예외 처리 — 오류가 나도 안 죽는 튼튼한 프로그램!

작성 파일 저장

에러 메시지 캡처

입력값 실험

다음 실습 준비

TAKEAWAY

파일 실습 전후에는 생성된 파일과 현재 폴더를 꼭 확인합니다.

3교시

3교시 — 예외 처리

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: try/except로 오류에 강한 프로그램을 만들고 디버깅한다.

이론 20분

실습 30분

휴식 10분

이론

프로그램이 죽는 순간

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 사용자가 예상 못한 입력을 하면 프로그램이 멈춤
- 0으로 나누기, 없는 파일 열기 등도 마찬가지
- 현실 프로그램은 이런 상황에도 버텨야 함!

Python example

```
age = int(input("나이: ")) # 사용자가 "스물" 입력  
# ValueError → 프로그램 즉시 중단!
```

TAKEAWAY

사용자가 예상 못한 입력을 하면 프로그램이 멈춤

이론

문법 오류 vs 예외

개념을 크게 잡고 바로 코드로 확인

핵심 개념

- 문법 오류(SyntaxError): 코드 자체가 틀림 → 아예 실행 안 됨
- 예: 콜론 빠뜨림, 괄호 안 닫음

적용 질문

- 예외(Exception): 실행 중 발생하는 오류 → 도중에 멈춤
- 예: 0으로 나누기, 글자를 숫자로 변환
- 예외는 대응(처리) 이 가능!

TAKEAWAY

문법 오류(SyntaxError): 코드 자체가 틀림 → 아예 실행 안 됨

이론

예외의 종류

개념을 크게 잡고 바로 코드로 확인

예외	발생 상황
ValueError	int("안녕") 변환 실패
ZeroDivisionError	10 / 0
FileNotFoundError	없는 파일 열기
KeyError	딕셔너리 없는 키
IndexError	리스트 범위 초과
TypeError	"3" + 5 타입 불일치

읽는 법

- 핵심 개념을 예시로 먼저 이해합니다.
- 코드로 직접 확인하며 감을 잡습니다.

TAKEAWAY

핵심 개념을 예시로 먼저 이해합니다.

이론

예외가 나면 어떻게 될까?

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 예외가 나면 그 줄에서 즉시 중단
- 뒤 코드는 실행되지 않음
- 사용자에게 빨간 에러 메시지만...

Python example

```
print("시작")  
print(10 / 0)    # 여기서 멈춤  
print("끝")      # 실행 안 됨!
```

TAKEAWAY

예외가 나면 그 줄에서 즉시 중단

이론

try - except 기본

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- try: 안에 위험한 코드
- except: 안에 오류 났을 때 할 일
- 오류가 나도 죽지 않고 except로!

Python example

```
try:
    age = int(input("나이: "))
    print(f"내년엔 {age + 1}살")
except:
    print("올바른 숫자를 입력하세요")
```

TAKEAWAY

try: 안에 위험한 코드

이론

try - except 흐름

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 오류 발생 → 즉시 except로 점프 → 이후 정상 진행

Python example

```
try:
    x = 10 / 0      # ① 오류 발생
    print("이 줄은 건너뛴") # ② 실행 안 됨
except:
    print("오류를 처리했어요") # ③ 여기로!
    print("프로그램 계속됨")   # ④ 정상 진행
```

TAKEAWAY

오류 발생 → 즉시 except로 점프 → 이후 정상 진행

이론

특정 예외만 잡기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 예외 종류별로 다른 대응
- except 예외이름: 으로 꼭 집어 처리

Python example

```
try:
    x = int(input("숫자: "))
    print(10 / x)
except ValueError:
    print("숫자가 아닙니다")
except ZeroDivisionError:
    print("0으로 못 나눕니다")
```

TAKEAWAY

예외 종류별로 다른 대응

이론

에러 메시지 활용 (as e)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- as e 로 실제 오류 내용을 받아 출력
- 디버깅·로그 기록에 유용

Python example

```
try:
    x = int("안녕")
except ValueError as e:
    print(f"오류 발생: {e}")
# 오류 발생: invalid literal for int()...
```

TAKEAWAY

as e 로 실제 오류 내용을 받아 출력

이론

다중 예외 한 번에

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 여러 예외를 괄호로 묶어 같이 처리
- 같은 대응이면 한 줄로 간단히

Python example

```
try:
    ...
except (ValueError, ZeroDivisionError):
    print("입력 오류입니다")
```

TAKEAWAY

여러 예외를 괄호로 묶어 같이 처리

이론

else 절

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- else = 오류가 안 났을 때 실행
- "성공했을 때만 할 일"을 분리

Python example

```
try:
    x = int(input("숫자: "))
except ValueError:
    print("숫자 아님")
else:
    print(f"입력 성공: {x}") # 오류 없을 때만!
```

TAKEAWAY

else = 오류가 안 났을 때 실행

이론

finally 절

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- finally = 무조건 실행 (성공·실패 상관없이)
- 파일 닫기, 정리 작업에 사용

Python example

```
try:
    f = open("data.txt")
    ...
except FileNotFoundError:
    print("파일 없음")
finally:
    print("작업 종료") # 오류든 아니든 항상 실행!
```

TAKEAWAY

finally = 무조건 실행 (성공·실패 상관없이)

이론

raise — 예외 일부러 발생

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- raise 로 내가 직접 예외를 일으킴
- 잘못된 값을 미리 막을 때 사용

Python example

```
def set_age(age):  
    if age < 0:  
        raise ValueError("나이는 0 이상이어야 합니다")  
    return age  
  
set_age(-5) # ValueError 발생
```

TAKEAWAY

raise 로 내가 직접 예외를 일으킴

이론

입력 검증에 활용

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 올바른 값이 들어올 때까지 반복 + 예외 처리
- 실전 프로그램의 표준 패턴!

Python example

```
while True:
    try:
        age = int(input("나이: "))
        if age < 0:
            raise ValueError
        break          # 정상 입력이면 탈출
    except ValueError:
        print("0 이상의 숫자를 다시 입력하세요")
```

TAKEAWAY

올바른 값이 들어올 때까지 반복 + 예외 처리

이론

디버깅 — 에러 메시지 읽기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 맨 아래 줄에 오류 종류와 원인
- line 5 → 몇 번째 줄에서 났는지
- 에러는 혼내는 게 아니라 위치+원인 힌트!

Python example

```
Traceback (most recent call last):  
  File "test.py", line 5, in <module>  
    print(10 / x)  
ZeroDivisionError: division by zero
```

TAKEAWAY

맨 아래 줄에 오류 종류와 원인

이론

디버깅 — print로 추적

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 막히면 중간 값을 print 해보기
- "여기까지 잘 오나?" 확인하며 범위 좁히기
- 가장 쉽고 강력한 디버깅 1단계

Python example

```
def calc(a, b):  
    print(f"디버그: a={a}, b={b}") # 값 확인  
    return a / b
```

TAKEAWAY

막히면 중간 값을 print 해보기

실습

안전한 나눗셈 계산기

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
while True:
    try:
        a = int(input("첫 숫자: "))
        b = int(input("둘째 숫자: "))
        print(f"결과: {a / b}")
        break
    except ValueError:
        # ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

안전한 파일 읽기

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
filename = input("열 파일 이름: ")
try:
    with open(filename, "r", encoding="utf-8")
        as f:
            print(f.read())
except FileNotFoundError:
    print(f'{filename} 파일이 없습니다.')
```

실습 포인트

- 1교시의 FileNotFoundError 를 드디어 처리!

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

예외 처리 (택1 이상)

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

따라 하기

- 안전한 정수 입력 함수: 숫자 아닐 때 다시 묻기
- 안전한 메뉴 선택: 1~3 외 입력 시 안내 후 재입력

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- 안전한 csv 읽기: 파일 없으면 안내, 있으면 평균 계산
- 회원가입 검증: 나이에 음수·문자 넣으면 막기 (raise)

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

휴식

휴식 10분

파일을 저장하고 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 클래스 — 데이터와 기능을 하나로 묶는 객체지향!

작성 파일 저장

에러 메시지 캡처

입력값 실험

다음 실습 준비

TAKEAWAY

파일 실습 전후에는 생성된 파일과 현재 폴더를 꼭 확인합니다.

4교시

4교시 — 클래스 기초 ①

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 클래스와 객체의 개념을 이해하고 직접 정의한다.

이론 20분

실습 30분

휴식 10분

이론

지금까지의 한계

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 관련된 데이터가 따로따로 흩어짐
- 데이터(변수)와 기능(함수)이 분리됨
- 해결: 클래스 — 묶어서 관리!

Python example

```
# 학생 한 명의 정보가 따로 흩어짐
```

```
name = "홍길동"
```

```
student_id = "2024001"
```

```
score = 90
```

```
# 학생이 100명이면? 변수가 300개??
```

TAKEAWAY

관련된 데이터가 따로따로 흩어짐

이론

객체지향이란?

개념을 크게 잡고 바로 코드로 확인

핵심 개념

- 객체지향 = 현실의 사물을 코드로 모델링
- 현실: 자동차는 속성(색, 속도)과 기능(달리기, 멈추기)을 가짐

적용 질문

- 코드에서도 속성+기능을 하나로 묶기 → 객체
- 자동차, 학생, 계좌... 모두 객체로 표현 가능

TAKEAWAY

객체지향 = 현실의 사물을 코드로 모델링

이론

절차지향 vs 객체지향

개념을 크게 잡고 바로 코드로 확인

	절차지향	객체지향
방식	순서대로 함수 실행	객체들이 협력
데이터	변수로 흩어짐	객체 안에 묶임
비유	요리 레시피 순서	요리사·재료·도구
적합	간단한 작업	크고 복잡한 프로그램

읽는 법

- 지금까지진 절차지향, 오늘부터 객체지향 맛보기!

TAKEAWAY

지금까지진 절차지향, 오늘부터 객체지향 맛보기!

이론

클래스 vs 인스턴스 ★

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 클래스 = 붕어빵 틀 (설계도)
- 인스턴스 = 붕어빵 (틀로 찍어낸 실제 물건)
- 틀(클래스) 하나로 붕어빵(인스턴스) 여러 개!

Python example

```
class Dog: ...      # 틀
my_dog = Dog()      # 붕어빵 1
your_dog = Dog()    # 붕어빵 2
```

TAKEAWAY

클래스 = 붕어빵 틀 (설계도)

이론

클래스 정의 — class

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- class + 클래스이름(첫 글자 대문자 관례) + 콜론
- 안에 메서드(클래스 안 함수)를 정의
- 아직 설계도만 그린 상태

Python example

```
class Dog:  
    def bark(self):  
        print("멍멍!")
```

TAKEAWAY

class + 클래스이름(첫 글자 대문자 관례) + 콜론

이론

인스턴스 만들기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 클래스이름() 으로 인스턴스 생성
- 인스턴스.메서드() 로 기능 호출
- 붕어빵 틀에서 붕어빵을 찍어낸 것!

Python example

```
class Dog:
    def bark(self):
        print("멍멍!")

my_dog = Dog()    # 인스턴스 생성!
my_dog.bark()     # 멍멍!
```

TAKEAWAY

클래스이름() 으로 인스턴스 생성

이론

속성(attribute)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 속성 = 객체가 가진 데이터(이름, 나이 ...)
- 인스턴스.속성 으로 접근
- 하지만 매번 일일이 붙이긴 불편... → `__init__`!

Python example

```
class Dog:
    pass

my_dog = Dog()
my_dog.name = "바둑이"    # 속성 부여
my_dog.age = 3
print(my_dog.name)      # 바둑이
```

TAKEAWAY

속성 = 객체가 가진 데이터(이름, 나이...)

이론

__init__ 생성자 ★

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- __init__ = 인스턴스 생성 시 자동 실행되는 메서드
- 속성을 초기 설정(initialize)
- 인스턴스 만들 때 값을 넘김

Python example

```
class Dog:
    def __init__(self, name, age):
        self.name = name    # 속성 초기화
        self.age = age

my_dog = Dog("바둑이", 3)  # 생성 시 자동 호출!
print(my_dog.name)        # 바둑이
```

TAKEAWAY

__init__ = 인스턴스 생성 시 자동 실행되는 메서드

이론

self의 의미



개념을 크게 잡고 바로 코드로 확인

개념 포인트

- self = 그 인스턴스 자기 자신
- self.name = "이 객체의 name"
- my_dog의 self는 my_dog, your_dog의 self...

Python example

```
class Dog:
    def __init__(self, name):
        self.name = name    # self = 자기 자신

my_dog = Dog("바둑이")
your_dog = Dog("뽕뽕")
```

TAKEAWAY

self = 그 인스턴스 자기 자신

이론

self 동작 원리

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 메서드 호출 시 인스턴스가 self로 자동 전달
- 그래서 메서드 첫 매개변수는 항상 self
- 우리가 직접 넣지 않아도 자동!

Python example

```
my_dog = Dog("바둑이")  
# 파이썬이 내부적으로: Dog.__init__(my_dog, "바둑이")  
# → self = my_dog 가 됨
```

TAKEAWAY

메서드 호출 시 인스턴스가 self로 자동 전달

이론

속성 접근과 수정

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 인스턴스.속성 으로 읽기.쓰기
- 각 인스턴스는 독립적인 속성을 가짐

Python example

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

d = Dog("바둑이", 3)
print(d.age)      # 3
d.age = 4         # 수정
print(d.age)      # 4
```

TAKEAWAY

인스턴스.속성 으로 읽기.쓰기

이론

메서드(method)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 메서드 = 클래스 안의 함수(객체의 기능)
- 메서드 안에서 self.속성 으로 자기 데이터 사용

Python example

```
class Dog:
    def __init__(self, name):
        self.name = name
    def bark(self):
        print(f'{self.name}: 멍멍!')

d = Dog("바둑이")
d.bark()      # 바둑이: 멍멍!
```

TAKEAWAY

메서드 = 클래스 안의 함수(객체의 기능)

이론

메서드가 속성을 활용

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 메서드가 자기 속성(self.score)으로 판단·계산
- 데이터와 기능이 한 몸!

Python example

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score
    def is_pass(self):
        return self.score >= 60

s = Student("홍길동", 75)
print(s.is_pass()) # True
```

TAKEAWAY

메서드가 자기 속성(self.score)으로 판단·계산

이론

여러 인스턴스

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 하나의 클래스로 여러 인스턴스, 각자 독립
- 붕어빵 틀 하나로 붕어빵 여러 개!

Python example

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

s1 = Student("홍길동", 90)
s2 = Student("김철수", 85)
print(s1.name, s1.score) # 홍길동 90
print(s2.name, s2.score) # 김철수 85
```

TAKEAWAY

하나의 클래스로 여러 인스턴스, 각자 독립

정리

클래스 정리

개념을 크게 잡고 바로 코드로 확인

오늘의 결과물

- 클래스 = 설계도(틀), 인스턴스 = 실물(붕어빵)
- class 이름: + 메서드들
- `__init__(self, ...)` = 생성자 (속성 초기화)
- `self` = 자기 자신, `self.속성` 으로 데이터

다음 연결

- 인스턴스 = `클래스()`, `인스턴스.메서드()`

TAKEAWAY

데이터는 저장되고, 오류에도 버티고, 코드는 사물처럼 구조화됩니다.

실습

Student 클래스

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
class Student:
    def __init__(self, name, student_id,
                 score):
        self.name = name
        self.student_id = student_id
        self.score = score
    def show(self):
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

Car 클래스

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
class Car:
    def __init__(self, model):
        self.model = model
        self.speed = 0
    def accelerate(self):
        self.speed += 10
        print(f'{self.model} 가속! 현재
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

나만의 클래스 (택1 이상)

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

따라 하기

- Cat 클래스: 이름·나이 속성, meow() 메서드
- Phone 클래스: 배터리 속성, use()(감소)·charge()(충전)

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- Character 클래스: 체력·레벨, attack()·level_up()
- Book 클래스: 제목·저자·페이지, info() 출력 메서드

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

휴식

휴식 10분

파일을 저장하고 다음 개념으로 넘어갈 준비

10 MIN BREAK

다음 교시: 클래스 기초 ② — 메서드를 더 활용하
고 실전 클래스를!

작성 파일 저장

에러 메시지 캡처

입력값 실험

다음 실습 준비

TAKEAWAY

파일 실습 전후에는 생성된 파일과 현재 폴더를 꼭 확인합니다.

5교시

5교시 — 클래스 기초 ②

시간 구성: 이론 20분 → 실습 30분 → 휴식 10분 목표: 메서드를 활용
해 상태를 관리하는 실전 클래스를 만든다.

이론 20분

실습 30분

휴식 10분

복습

복습 — 클래스 구조

개념을 크게 잡고 바로 코드로 확인

지난주까지

- 변수·조건·반복·자료구조·함수·문자열로 기초 문법을 완성했습니다.
- 하지만 프로그램을 고면 데이터가 사라지고, 잘못된 입력에는 쉽게 멈춥니다.

오늘의 해결

- 파일 입출력: 데이터를 영구 저장
- 예외 처리: 오류에도 안 죽는 흐름
- 클래스: 데이터와 기능을 한 단위로 묶기

끝나면 만들 수 있는 것

TAKEAWAY

저장·안정성·구조화는 실전 프로그램으로 가는 세 갈래 문입니다.

이론

메서드로 상태 바꾸기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 메서드가 속성(상태)을 변경
- 객체가 자신의 상태를 기억하고 관리

Python example

```
class Counter:
    def __init__(self):
        self.count = 0
    def increase(self):
        self.count += 1
    def reset(self):
        self.count = 0
c = Counter()
c.increase(); c.increase()
print(c.count) # 2
```

TAKEAWAY

메서드가 속성(상태)을 변경

이론

기본값 속성

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- `__init__` 에도 기본값 매개변수 사용 가능 (3일차 복습)
- 안 넘기면 기본값 적용

Python example

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance # 기본 잔액 0

a = BankAccount("홍길동") # 잔액 0
b = BankAccount("김철수", 10000) # 잔액 10000
```

TAKEAWAY

`__init__` 에도 기본값 매개변수 사용 가능 (3일차 복습)

이론

메서드가 다른 메서드 호출

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 메서드 안에서 self.다른메서드() 호출 가능
- 기능을 작게 쪼개 조립!

Python example

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score
    def grade(self):
        return "합격" if self.score >= 60 else
            "불합격"
    def report(self):
        print(f"{self.name}: {self.score}점
            ({self.grade()})")
```

TAKEAWAY

메서드 안에서 self.다른메서드() 호출 가능

이론

메서드 vs 일반 함수

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 함수는 값을 받아 처리, 메서드는 자기 속성(self)으로 처리
- 메서드는 객체에 소속된 함수

Python example

```
# 일반 함수 (3일차)
def is_pass(score):
    return score >= 60
is_pass(75)          # 값을 직접 넘김

# 메서드 (클래스 안)
class Student:
    def is_pass(self):
        return self.score >= 60
s.is_pass()          # 객체가 자기 데이터로!
```

TAKEAWAY

함수는 값을 받아 처리, 메서드는 자기 속성(self)으로 처리

이론

__str__ — 객체를 예쁘게 출력

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- __str__ 정의 → print(객체) 시 보기 좋게 출력
- 안 하면 <__main__.Student object at 0x...

Python example

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score
    def __str__(self):
        return f"{self.name}{self.score}점"

s = Student("홍길동", 90)
print(s)      # 홍길동(90점) ← __str__ 덕분!
```

TAKEAWAY

__str__ 정의 → print(객체) 시 보기 좋게 출력

이론

객체를 리스트에 담기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 객체도 리스트에 담아 여러 개 관리!
- 어제 자료구조 + 오늘 클래스 = 강력한 조합

Python example

```
students = [  
    Student("홍길동", 90),  
    Student("김철수", 85),  
    Student("이영희", 95),  
]  
for s in students:  
    print(s)    # __str__로 예쁘게
```

TAKEAWAY

객체도 리스트에 담아 여러 개 관리!

이론

객체 리스트 다루기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 객체 리스트에 `sum·max` 등 그대로 활용

Python example

```
students = [Student("홍길동", 90), Student("김철수", 85)]
```

```
# 평균 점수
```

```
total = sum(s.score for s in students)  
print(f'평균: {total / len(students):.1f}')
```

```
# 최고 점수 학생
```

```
top = max(students, key=lambda s: s.score)  
print(f'1등: {top.name}')
```

TAKEAWAY

객체 리스트에 `sum·max` 등 그대로 활용

이론

객체를 함수에 넘기기

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 객체도 변수처럼 함수에 넘길 수 있음
- 함수 안에서 객체.속성.객체.메서드() 사용

Python example

```
def print_report(student):    # 객체를 매개변수로
    print(f"{student.name}: {student.score}점")

def is_top(student):
    return student.score >= 90

s = Student("홍길동", 95)
print_report(s)
print(is_top(s))            # True
```

TAKEAWAY

객체도 변수처럼 함수에 넘길 수 있음

이론

인스턴스 변수 vs 클래스 변수 (맛보기)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 인스턴스 변수(self.) = 객체마다 다름
- 클래스 변수 = 모든 객체가 공유
- 깊게는 x, "이런 구분이 있다" 정도

Python example

```
class Dog:
    species = "개"      # 클래스 변수 (모두 공유)
    def __init__(self, name):
        self.name = name # 인스턴스 변수 (각자)

a = Dog("바둑이")
print(a.name, a.species) # 바둑이 개
```

TAKEAWAY

인스턴스 변수(self.) = 객체마다 다름

이론

클래스 + 예외 처리 결합

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 메서드 안에서 검증 + raise (3교시 결합)
- 잘못된 동작을 객체 스스로 막음

Python example

```
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance
    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("잔액 부족!")
        self.balance -= amount
```

TAKEAWAY

메서드 안에서 검증 + raise (3교시 결합)

이론

클래스 + 파일 결합 (예고)

개념을 크게 잡고 바로 코드로 확인

개념 포인트

- 객체 데이터를 파일로 저장 → 꺼도 안 사라짐!
- 오늘 배운 3가지(파일+예외+클래스)가 한 곳에

Python example

```
import json
class Diary:
    def __init__(self):
        self.entries = []
    def save(self):
        with open("diary.json", "w",
                  encoding="utf-8") as f:
            json.dump(self.entries, f,
                      ensure_ascii=False)
```

TAKEAWAY

객체 데이터를 파일로 저장 → 꺼도 안 사라짐!

이론

클래스 설계 팁

개념을 크게 잡고 바로 코드로 확인

핵심 개념

- 하나의 클래스 = 하나의 사물/개념 (Student, Car, Account)
- 속성: 그 사물이 가진 데이터

적용 질문

- 메서드: 그 사물이 할 수 있는 일
- 명사는 클래스, 형용사·정보는 속성, 동사는 메서드

TAKEAWAY

하나의 클래스 = 하나의 사물/개념 (Student, Car, Account)

이론

클래스 흔한 실수

개념을 크게 잡고 바로 코드로 확인

핵심 개념

- self 빼먹기: `def bark():` → `def bark(self):`
- 속성 앞에 self 안 붙임: `name = name` → `self.name = na...`

적용 질문

- 인스턴스 생성 시 () 빠뜨림: `Dog` → `Dog()`
- `__init__` 밑줄 개수 틀림 (양쪽 2개씩)

TAKEAWAY

self 빼먹기: `def bark():` → `def bark(self):`

정리

오늘 총정리 — 3개의 무기

개념을 크게 잡고 바로 코드로 확인

오늘의 결과물

- 파일 입출력: 데이터를 영구 저장 (꺼도 안 사라짐)
- 예외 처리: 오류에도 안 죽는 튼튼함
- 클래스: 데이터+기능을 사물 단위로 묶기
- 이 셋을 합치면 → 진짜 프로그램! (미니 프로젝트!)

다음 연결

- 상속으로 클래스를 확장합니다.
- 매직 메서드로 객체 표현을 다듬습니다.
- 외부 라이브러리로 데이터·그래프·웹을 다룹니다.

TAKEAWAY

데이터는 저장되고, 오류에도 버티고, 코드는 사물처럼 구조화됩니다.

실습

BankAccount 클래스

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
        print(f'{amount:,}원 입금 → 잔액
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

여러 계좌 관리

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

2. 일부러 틀려보기

3. 결과 설명

실습 코드

```
accounts = [  
    BankAccount("홍길동", 10000),  
    BankAccount("김철수", 50000),  
    BankAccount("이영희", 30000),  
]  
# 전체 잔액 합계  
total = sum(acc.balance for acc in accounts)  
# ... 계속
```

실습 포인트

- 한 줄씩 실행 결과를 예측합니다.
- 입력값을 바꿔 여러 경우를 확인합니다.

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

실습

실전 클래스 (택1 이상)

실행해 보고, 일부러 틀려 보고, 결과를 확인하는 실습

1. 따라 치기

따라 하기

- ShoppingCart: 상품 추가/삭제(리스트), 총액 계산
메서드
- Playlist: 곡 추가, 다음 곡 재생, 목록 출력

2. 일부러 틀려보기

3. 결과 설명

변형 미션

- TodoList: 할 일 추가/완료 처리/남은 일 개수
- Pet: 배고픔·행복 속성, feed()·play()로 상태 변화

TAKEAWAY

파일·예외·클래스는 직접 저장하고 일부러 실패시킬 때 빠르게 익숙해집니다.

휴식

오프라인 정리 & 휴식

잠깐 쉬고, 온라인에서는 이 셋을 합쳐 "진짜 저장되는 프로그램"을 만듭니다!

WRAP-UP

파일 입출력(open/with, r/w/a) · CSV · JSON

작성 파일 저장

에러 메시지 캡처

입력값 실험

다음 실습 준비

TAKEAWAY

파일 실습 전후에는 생성된 파일과 현재 폴더를 꼭 확인합니다.